

Bermudan Option Pricing

Polynomial and Neural Network Regression

Mohamed Guendzi

September 2026

Contents

1	Introduction	2
2	Mathematical framework	2
2.1	Market model and exercise payoffs	2
2.2	Multiple assets and exact simulation	3
2.3	Optimal stopping and the Snell envelope	3
2.4	Conditional expectation and least squares	4
2.5	Polynomial regression	4
3	The Longstaff-Schwartz method	5
3.1	Backward regression and exercise decisions	5
3.2	Independent policy evaluation	6
4	Neural continuation regression	7
4.1	Replacing the polynomial approximation	7
4.2	Architecture and dimensions	7
4.3	Parameter count and comparison criteria	8
4.4	Activation and scaling	9
4.5	Mini-batches, Adam and warm starts	9
5	Implementation and experimental protocol	10
5.1	Code organization and numerical checks	10
5.2	Contracts and reference values	11
5.3	Independent comparisons and uncertainty	12
6	Numerical results	13
6.1	Checks and reference values	13
6.2	Policy values with 10^5 training paths	13
6.3	Training estimates and training effort	14
6.4	Effect of the training budget	15
7	Conclusion	17
	References	18

1 Introduction

Bermudan option pricing requires an exercise policy as well as a valuation procedure. At each permitted date, the holder compares the immediate payoff with the conditional value of continuing. Longstaff–Schwartz estimates this continuation by regression on simulated paths, allowing different approximation families to be used within the same stopping recursion.

The aim of this project is to compare polynomial and neural network regressors under a common simulation and evaluation protocol. The comparison concerns the values of the learned exercise policies on independent paths, together with numerical accuracy and computational cost. Polynomial bases provide a transparent least-squares baseline; neural networks learn their features and allow the approximation budget to be controlled through hidden-layer width. The study separates approximation, training and evaluation, so that a regression prediction is never confused with either a realized cashflow or a guaranteed option price.

2 Mathematical framework

2.1 Market model and exercise payoffs

Consider one asset under the risk-neutral measure $\mathbb{Q}$:

$$dS_t = (r - \delta)S_t dt + \sigma S_t dW_t^{\mathbb{Q}}, \quad S_0 > 0. \quad (1)$$

The interest rate $r \geq 0$, continuous dividend yield $\delta \geq 0$ and volatility $\sigma > 0$ are constant. The asset pays dividends at rate $\delta S_t dt$; its total expected return under $\mathbb{Q}$, including dividends, is r . For a uniform exercise grid $t_n = n\Delta t$, $N\Delta t = T$, the exact transition is

$$S_{t_{n+1}} = S_{t_n} \exp\left[(r - \delta - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\xi_n\right], \quad \xi_n \sim \mathcal{N}(0, 1), \quad (2)$$

with independent increments across dates and independent simulated paths. This avoids Euler discretization error at the prescribed exercise dates.

A Bermudan option may be exercised at $t_0 = 0, t_1, \dots, t_N = T$. For a put with strike K , the exercise payoff is $g(s) = (K - s)^+$. Throughout the report, all exercise cashflows are discounted to time zero:

$$Z_n = e^{-rt_n} g(S_{t_n}). \quad (3)$$

This convention allows direct comparisons between cashflows paid at different dates.

Dividends and early exercise. For a call without dividends, waiting until maturity has value at least

$$\mathbb{E}^{\mathbb{Q}}[e^{-r(T-t)}(S_T - K)^+ | \mathcal{F}_t] \geq S_t - Ke^{-r(T-t)} \geq S_t - K.$$

The waiting value is also nonnegative, so it dominates immediate exercise. Thus, for $r \geq 0$, a non-dividend-paying call gains no value from early exercise. With dividends, the first lower bound becomes $S_te^{-\delta h} - Ke^{-rh}$, where $h = T - t$. Its difference from $S_t - K$ is

$$K(1 - e^{-rh}) - S_t(1 - e^{-\delta h}),$$

which can be negative: delaying payment of the strike must now be weighed against dividends forgone while holding the option. Early exercise can become worthwhile. For a put, dividends

are not required: exercising receives the strike earlier, which can be beneficial when rates are positive.

2.2 Multiple assets and exact simulation

For d assets, one state is the vector $S_t = (S_t^1, \dots, S_t^d)$. With constant parameters, each component satisfies

$$dS_t^j = (r - \delta_j)S_t^j dt + \sigma_j S_t^j dW_t^j, \quad d\langle W^i, W^j \rangle_t = \Gamma_{ij} dt.$$

Here Γ is a positive definite correlation matrix. If $LL^\top = \Gamma$ and $\xi_n \sim \mathcal{N}(0, I_d)$, an exact joint transition is

$$S_{t_{n+1}}^j = S_{t_n}^j \exp\left[(r - \delta_j - \frac{1}{2}\sigma_j^2)\Delta t + \sigma_j \sqrt{\Delta t} (L\xi_n)_j\right]. \quad (4)$$

Gaussian vectors are independent across dates and simulated paths; within a vector, multiplication by L produces the required asset correlations. The step $\Delta t = T/N$ connects exercise dates. There is no Euler step to refine: the transition law on this grid is exact. Adding exercise dates changes the Bermudan contract rather than correcting an Euler approximation.

Paths, features and storage. M paths of a d -asset model means M joint market scenarios, each containing d asset prices at each date. Including the initial state, the path array has shape $(M, N + 1, d)$. At one date it supplies an input matrix with shape (M, d) , not Md training observations. With $M = 10^6$ and $d = 50$, this is one million observations of fifty features. There are fifty million scalar prices per date; storing ten future dates in float32 requires about 2 GB for those prices alone. Memory requirements grow linearly with the number of paths, dates and assets.

The payoff remains scalar, for example $g(s) = (\max_j s_j - K)^+$ for a max-call or $g(s) = (K - (\prod_j s_j)^{1/d})^+$ for a geometric basket put. The continuation is consequently a function from $\mathbb{R}^d$ to $\mathbb{R}$.

2.3 Optimal stopping and the Snell envelope

Let $\mathcal{F}_n = \mathcal{F}_{t_n}$ denote the information available at date t_n . An exercise rule determines a random index $\tau \in \{0, \dots, N\}$; the corresponding calendar date is t_τ . It is admissible if

$$\{\tau \leq n\} \in \mathcal{F}_n \quad \text{for every } n. \quad (5)$$

This is the stopping-time condition: by date t_n , the available information must determine whether exercise has already occurred. For example, exercising at the first permitted date when $S_{t_n} \leq b_n$, for predetermined thresholds b_n , and otherwise at maturity, defines an admissible rule. Different paths can produce different exercise dates under the same rule. At maturity, a zero payoff simply represents worthless expiry.

Let $\mathcal{T}_n$ be the stopping times taking values in $\{n, \dots, N\}$. The time-zero value in the model is

$$V_0 = \sup_{\tau \in \mathcal{T}_0} \mathbb{E}^\mathbb{Q}[Z_\tau]. \quad (6)$$

This optimizes expected discounted payoff over feasible rules. It is a model value, not an observed market quotation, and its definition does not presume that it is available in closed form.

Continuation values. At the penultimate date, waiting leaves only the terminal payoff. Since the asset is Markov, the relevant continuation function is

$$c_{N-1}^*(x) = \mathbb{E}^{\mathbb{Q}}[Z_N \mid S_{t_{N-1}} = x]. \quad (7)$$

For a put observed at $S_{t_{N-1}} = 100$ with $K = 110$, the comparison is between $10e^{-rt_{N-1}}$ and $c_{N-1}^*(100)$. Conditioning on the current price accounts for the information available when making the decision.

The same comparison extends backwards through the Snell envelope:

$$U_N = Z_N, \quad U_n = \max\{Z_n, \mathbb{E}^{\mathbb{Q}}[U_{n+1} \mid \mathcal{F}_n]\}. \quad (8)$$

Here U_n is the optimal conditional value from date t_n , still expressed in time-zero units, and $U_0 = V_0$ when the initial state is fixed. Writing $c_n^*(S_{t_n}) = \mathbb{E}^{\mathbb{Q}}[U_{n+1} \mid \mathcal{F}_n]$, an optimal rule is

$$\tau^* = \min(\{n < N : Z_n \geq c_n^*(S_{t_n})\} \cup \{N\}). \quad (9)$$

The future realization remains unknown. Optimality means maximizing its conditional expectation, not guaranteeing the largest realized payoff. Computing the continuation functions is the numerical challenge addressed by regression.

2.4 Conditional expectation and least squares

Fix a date and a future exercise policy. Let X be the current state and Y the discounted cashflow obtained by continuing and following that policy. At the penultimate date, $X = S_{t_{N-1}}$ and $Y = Z_N$. The function to estimate is

$$c(x) = \mathbb{E}^{\mathbb{Q}}[Y \mid X = x]. \quad (10)$$

The random variable Y is a realized future cashflow; $c(X)$ is its conditional mean. Simulations provide observations $(X^{(m)}, Y^{(m)})$, $m = 1, \dots, M$. A global average of the targets estimates $\mathbb{E}^{\mathbb{Q}}[Y]$, not the state-dependent continuation. With a continuous state distribution, a finite sample almost surely contains no observation exactly equal to a specified x . Regression uses observations across states to estimate a function that can be evaluated at x .

The L^2 projection. Assume $Y \in L^2$. For every square-integrable function $f(X)$,

$$\mathbb{E}^{\mathbb{Q}}[(Y - f(X))^2] = \mathbb{E}^{\mathbb{Q}}[(Y - c(X))^2] + \mathbb{E}^{\mathbb{Q}}[(c(X) - f(X))^2]. \quad (11)$$

Indeed, write $Y - f(X) = (Y - c(X)) + (c(X) - f(X))$ and expand. The cross term vanishes because $\mathbb{E}^{\mathbb{Q}}[Y - c(X) \mid X] = 0$. Thus $c(X)$ is the orthogonal projection of Y onto $L^2(\sigma(X))$. Minimizing squared error on noisy cashflows targets their conditional mean; it does not require each individual future cashflow to be predictable.

2.5 Polynomial regression

Choose basis functions $\phi_1, \dots, \phi_q$ and write $f_\beta(x) = \sum_{j=1}^q \beta_j \phi_j(x)$. For one asset and degree two, $\phi(x) = (1, x, x^2)^\top$ and $q = 3$. The approximation is linear in its coefficients, although it is nonlinear in x . The population problem minimizes $\mathbb{E}^{\mathbb{Q}}[(Y - f_\beta(X))^2]$; its empirical version is

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^q} \frac{1}{M} \sum_{m=1}^M (Y^{(m)} - f_\beta(X^{(m)}))^2. \quad (12)$$

Define the design matrix $\Phi_{mj} = \phi_j(X^{(m)})$ and target vector $y = (Y^{(1)}, \dots, Y^{(M)})^\top$. Then

$$\underbrace{\Phi}_{M \times q} \underbrace{\beta}_{q \times 1}, \quad \hat{\beta} = \arg \min_{\beta} \frac{1}{M} \|y - \Phi\beta\|^2, \quad \Phi^\top \Phi \hat{\beta} = \Phi^\top y. \quad (13)$$

The coefficients are unique if Φ has full column rank. The fitted vector is the Euclidean projection of y onto the column space of Φ . There are q coefficients per date, shared by all paths. In d dimensions, a total-degree- p basis contains $q = \binom{d+p}{p}$ terms, including the constant. For degree two, this counts one constant, d linear terms and $d(d+1)/2$ quadratic terms. At $d = 50$, degrees two and three require respectively 1326 and 23426 coefficients per date. These are basis sizes, not asset counts or numbers of paths. Their growth motivates comparing other function families.

3 The Longstaff–Schwartz method

3.1 Backward regression and exercise decisions

Longstaff–Schwartz [1] implements a backward recursion on exercise policies. At date t_n , the future policy has already been learned. Let $\hat{\tau}_{n+1}$ denote the exercise index obtained by following it from t_{n+1} onwards. The regression target is

$$X_n = S_{t_n}, \quad Y_n = Z_{\hat{\tau}_{n+1}}. \quad (14)$$

Each date has its own target and fitted function $\hat{c}_n$. If $\mathcal{D}$ denotes the training data, the conditional mean being approximated on a fresh path is

$$c_n^{\mathcal{D}}(x) = \mathbb{E}^{\mathbb{Q}}[Z_{\hat{\tau}_{n+1}} \mid S_{t_n} = x, \mathcal{D}].$$

This is the continuation of the learned future policy, which need not coincide with the optimal continuation c_n^* .

Backward recursion. Simulate M training paths and initialize $Y_{N-1}^{(m)} = Z_N^{(m)}$. For $n = N-1, \dots, 1$, fit a regressor on the in-the-money paths $I_n = \{m : Z_n^{(m)} > 0\}$:

$$\hat{\beta}_n \in \arg \min_{\beta} \frac{1}{|I_n|} \sum_{m \in I_n} (Y_n^{(m)} - f_{\beta}(S_{t_n}^{(m)}))^2, \quad \hat{c}_n = f_{\hat{\beta}_n}. \quad (15)$$

The fit is skipped if there are no eligible observations. For a finite polynomial basis, the sample size and numerical rank also require attention. After fitting, form the target for the preceding date:

$$Y_{n-1}^{(m)} = \begin{cases} Z_n^{(m)}, & m \in I_n \text{ and } Z_n^{(m)} \geq \hat{c}_n(S_{t_n}^{(m)}), \\ Y_n^{(m)}, & \text{otherwise.} \end{cases} \quad (16)$$

No additional discount factor is needed in this update. The in-the-money restriction concentrates the fit where exercise can yield a positive payoff. It changes the distribution under which the finite-basis approximation is fitted. When $Z_n = 0$, continuing has nonnegative conditional value and is at least as good as exercising. We therefore continue in such states, including at equality. An unconstrained polynomial can predict a negative continuation even when all targets are nonnegative; the explicit condition $Z_n > 0$ prevents such a prediction from triggering worthless exercise. The retained target is then $Y_{n-1} = Y_n$.

A three-date example. For one training path with exercise dates $t_1, t_2, t_3 = T$, suppose the discounted payoffs are $Z_1 = 6$, $Z_2 = 12$, $Z_3 = 18$. The regression at t_2 uses target $Y_2 = 18$. Two possible fits illustrate how the next target is formed:

$\hat{c}_2(S_{t_2})$	Decision at t_2	Target Y_1	Input at t_1
11	Exercise	12	S_{t_1}
15	Continue	18	S_{t_1}

The prediction determines the decision; the target is the realized cashflow under that decision. In particular, it is not $\max\{Z_2, \hat{c}_2(S_{t_2})\}$. The same training paths are reused backwards, so these adaptively constructed targets do not constitute independent regression datasets across dates.

3.2 Independent policy evaluation

The backward pass produces one fitted function per exercise date. At time zero, where the initial state is fixed, compare Z_0 with the training estimate of continuing, $M^{-1} \sum_m Y_0^{(m)}$, and freeze that decision as well. All paths begin at the same deterministic S_0 : a sample average suffices to estimate continuation there, with no additional state-dependent regression. The exercise-versus-continuation comparison itself is unchanged. For paths on which time-zero exercise is not chosen, the subsequent rule is

$$\hat{\tau}_{\mathcal{D}} = \min(\{1 \leq n < N : Z_n > 0, Z_n \geq \hat{c}_n(S_{t_n})\} \cup \{N\}). \quad (17)$$

Given the learned functions, this rule uses only currently available information. It is applied forwards in time, on new paths independent of $\mathcal{D}$.

Write $H^{(m)} = Z_{\hat{\tau}_{\mathcal{D}}}^{(m)}$ for the resulting discounted test cashflows. The estimated policy value is

$$\hat{V} = \frac{1}{M_{\text{test}}} \sum_{m=1}^{M_{\text{test}}} H^{(m)}. \quad (18)$$

The averaged quantities are realized cashflows, not continuation predictions. Conditionally on the training data, this estimates

$$V(\hat{\tau}; \mathcal{D}) = \mathbb{E}^{\mathbb{Q}}[Z_{\hat{\tau}_{\mathcal{D}}} | \mathcal{D}] \leq V_0. \quad (19)$$

The inequality follows because the learned policy is one admissible rule among those defining V_0 . It concerns the policy's expectation: a finite test mean can exceed V_0 through sampling variability. The dispersion of independent test cashflows quantifies Monte Carlo uncertainty for this fixed policy, but not its shortfall from the optimum. Two learned policies can therefore be compared without knowing V_0 : the one with the higher value is closer to it. This ordering applies to values estimated on independent paths, up to Monte Carlo error, not to estimates computed on training paths.

Regression error and policy value. A continuation estimate may be too high or too low. For a fixed future policy, its error causes a loss at the current date only if it changes the exercise decision. The relevant comparison is between expected cashflows under the two decisions; the predicted continuation is never itself paid.

Optimism of training estimates. On a training path, the target $Y_n^{(m)}$ contains the future cashflow of that same path. A flexible regression partly fits this noise, so that $\hat{c}_n(X_n^{(m)})$ is pulled towards the realized future of the path: decisions taken on training paths use some information about the future. In the extreme case of exact interpolation, $\hat{c}_n(X_n^{(m)}) = Y_n^{(m)}$ at every date, the rule exercises when $Z_n^{(m)} \geq Y_n^{(m)}$, and the backward recursion gives $Y_0^{(m)} = \max_{1 \leq n \leq N} Z_n^{(m)}$. With $\bar{Y}_0 = M^{-1} \sum_m Y_0^{(m)}$, the training estimate $\max(Z_0, \bar{Y}_0)$ then satisfies

$$\mathbb{E}^{\mathbb{Q}}[\max(Z_0, \bar{Y}_0)] \geq \max\left(Z_0, \mathbb{E}^{\mathbb{Q}}\left[\max_{1 \leq n \leq N} Z_n\right]\right) \geq \max\left(Z_0, \sup_{\tau \in \mathcal{T}_1} \mathbb{E}^{\mathbb{Q}}[Z_\tau]\right) = V_0,$$

by Jensen's inequality for the convex map $y \mapsto \max(Z_0, y)$, and because $\mathcal{F}_0$ is trivial. After time zero, the training average then estimates the value of exercise decisions taken with knowledge of the whole path. This mechanism disappears on new paths, where the fitted noise carries no information. How large it is depends on the regression and on the sample, and is not determined by this argument alone. Relative to V_0 , a training estimate can therefore be biased in either direction: downwards because the learned policy is suboptimal, upwards because of this partial foresight. Relative to the value of the same policy on new paths, only the foresight effect remains, since both evaluate the same rule. Independent evaluation separates learning the rule from measuring its value. If candidate policies are selected using validation paths, the final test paths must also be independent of that selection.

4 Neural continuation regression

4.1 Replacing the polynomial approximation

At each date, the inputs and targets remain $(X_n^{(m)}, Y_n^{(m)})$. Polynomial regression chooses coefficients for a fixed basis of monomials. Neural regression instead learns weights and biases in a network taking the current asset prices as inputs. It does not estimate polynomial coefficients and does not require the polynomial feature matrix.

Write f_θ for the network before fitting. Without scaling, after training at date t_n , $\hat{c}_n = f_{\hat{\theta}_n}$ is the fitted continuation function. Its value $\hat{c}_n(x)$ is scalar. Both regressors target the same conditional mean through empirical squared error on the eligible paths:

$$J_n(\theta) = \frac{1}{|I_n|} \sum_{m \in I_n} \left(Y_n^{(m)} - f_\theta(X_n^{(m)})\right)^2. \quad (20)$$

The L^2 identity still applies, but fitting the network is generally a nonconvex optimization problem. Training does not guarantee the global minimum. During a fit, the targets are held fixed; gradients do not propagate through the exercise decisions or the backward stopping recursion.

4.2 Architecture and dimensions

Use the row-vector convention for an observation $x \in \mathbb{R}^{1 \times d}$. For one hidden layer with h neurons, the forward computation is

$$\begin{aligned} a^{[0]} &= x, & z^{[1]} &= a^{[0]}W^{[1]} + b^{[1]}, \\ a^{[1]} &= g^{[1]}(z^{[1]}), & \hat{y} &= a^{[2]} = a^{[1]}W^{[2]} + b^{[2]}. \end{aligned} \quad (21)$$

The hidden activation acts componentwise; the output activation is the identity. A continuation is not a class probability, so no sigmoid is used at the output. Superscripts in square brackets label network layers, while n labels exercise dates. Network pre-activations $z^{[\ell]}$ are distinct from discounted payoffs Z_n .

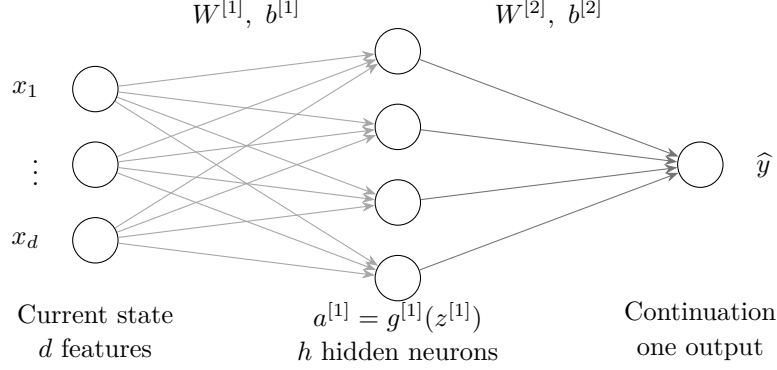


Figure 1: One observation through the continuation network. Only representative neurons are drawn. All observations use the same weights at a given exercise date. In the implementation, the input and output use the scaled units defined below.

Several observations. Stack the M observations in $A^{[0]} \in \mathbb{R}^{M \times d}$. The same computation becomes

$$Z^{[1]} = A^{[0]}W^{[1]} + b^{[1]}, \quad A^{[1]} = g^{[1]}(Z^{[1]}), \quad \hat{Y} = A^{[1]}W^{[2]} + b^{[2]},$$

with biases broadcast across rows. The dimensions are

Quantity	Shape	Role
$A^{[0]}$	$M \times d$	One row per observation
$W^{[1]}, b^{[1]}$	$d \times h, 1 \times h$	Hidden affine map
$Z^{[1]}, A^{[1]}$	$M \times h$	Hidden pre-activations and activations
$W^{[2]}, b^{[2]}$	$h \times 1, 1 \times 1$	Output affine map
$\hat{Y}$	$M \times 1$	One continuation prediction per row

The sample size changes the number of rows processed, not the weight dimensions. For $d = 50$ and $h = 32$, $W^{[1]}$ has shape $(50, 32)$ and $W^{[2]}$ has shape $(32, 1)$, whether training uses a thousand or a million paths. Here M denotes the total number of observations and m indexes one observation. When fitting only on in-the-money paths at date t_n , the input has $|I_n|$ rows.

4.3 Parameter count and comparison criteria

There are

$$dh + h + h + 1 = (d + 2)h + 1$$

trainable parameters per fitted network. At $d = 50$, a width of 32 gives 1665 parameters: more than a quadratic polynomial's 1326, but fewer than a cubic polynomial's 23426. Hidden width is a separate choice from polynomial degree. The jump from quadratic to cubic polynomial bases motivates testing a neural alternative with a controlled width. At fixed width, its parameter count grows linearly with d , and its hidden features are learned rather than prescribed. This

offers a different approximation budget, without guaranteeing a better policy or a shorter training time.

The fitted networks enter the same policy evaluation as the polynomials. Independent policy value, its uncertainty and stability across repetitions remain the comparison criteria; regression MSE alone does not rank policies.

4.4 Activation and scaling

Following the numerical experiments of Lapeyre–Lelong [2], the hidden activation is a leaky ReLU:

$$g^{[1]}(z) = \begin{cases} z, & z \geq 0, \\ 0.3z, & z < 0. \end{cases}$$

Ordinary ReLU has zero derivative on the negative half-line. A hidden neuron whose pre-activation is negative on every training observation contributes no gradient to its incoming weights. Leaky ReLU preserves a nonzero activation derivative there. This motivates its use, without establishing that it outperforms ReLU in this problem or that the slope 0.3 is optimal. The output remains linear; its predictions are not constrained to be nonnegative. The positive-payoff exercise condition in Section 3 still applies.

Both regression families use the fixed transformations

$$\tilde{x} = x/K - \mathbf{1}, \quad \tilde{Y}_n = Y_n/K, \quad \hat{c}_n(x) = Kf_{\hat{\theta}_n}(x/K - \mathbf{1}), \quad (22)$$

with θ replaced by β for the polynomial. Prices are centred at the strike and targets are expressed in strike units. This reduces dependence of optimization on the monetary units and brings near-the-money inputs closer to zero. It is not variance standardization, and does not guarantee well-conditioned features in every model. No statistics are re-estimated on validation or test data. Predictions are converted back to monetary units before exercise decisions. Scaling the target is distinct from discounting: Y_n was already discounted to time zero. For a full polynomial basis, the affine change of input preserves its span; the exact least-squares problem is only reparametrized.

4.5 Mini-batches, Adam and warm starts

At a fixed exercise date, the future-policy cashflows $Y_n^{(m)}$ are held fixed throughout training. The loss on a mini-batch $\mathcal{B} \subset I_n$ is

$$J_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{m \in \mathcal{B}} \left(f_{\theta}(\tilde{X}_n^{(m)}) - \tilde{Y}_n^{(m)} \right)^2.$$

Backpropagation computes its gradient. An optimizer then updates the parameters before the next batch. One epoch visits every eligible observation once, with observations shuffled between epochs. Thus an epoch has $\lceil |I_n|/B \rceil$ updates for batch size B , whereas full-batch descent has one. Here B denotes only the number of rows in a mini-batch; M remains the number of simulated paths. Mini-batches reduce memory for network activations and computation per update, at the cost of a noisier gradient.

Adam [3] combines a moving average of gradients with a moving average of their componentwise squares. If g_k is the mini-batch gradient at update k , its two accumulators, initially zero, satisfy

$$u_k = \beta_1 u_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^{\odot 2}.$$

After the initial-zero corrections $\hat{u}_k = u_k/(1 - \beta_1^k)$ and $\hat{v}_k = v_k/(1 - \beta_2^k)$, the update is

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{u}_k}{\sqrt{\hat{v}_k} + \varepsilon}.$$

All operations in the fraction are componentwise. The first average smooths the direction; the second adjusts the update scale for each parameter. The learning rate η is still a tuning choice. Adam neither changes the squared-error objective nor guarantees its global minimum. The implementation uses $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$.

When moving backwards from date $n + 1$ to n , the next network starts from the previously fitted weights. Each date retains a separate fitted model, and Adam’s accumulators are reset. The fixed scaling in (22) keeps the meaning of these weights consistent across dates. Nearby continuation functions may be similar, making such a warm start useful, but shorter training must be assessed numerically. Following Lapeyre–Lelong [2], a network trained from random weights, at the last regression date, uses ten epochs, and a warm-started network uses one epoch. The article reports that further passes did not reduce the training error at earlier dates in its experiments; a variant with five epochs after each warm start tests this in our setting. Widths are 32 and 128, with batch size 256 and learning rate 10^{-3} . These are predeclared candidates, not optimized hyperparameters, and not an exact replication of the article, which does not specify every training detail.

5 Implementation and experimental protocol

5.1 Code organization and numerical checks

The notebook separates market simulation, payoffs, regression and exercise decisions. A `BlackScholesModel` generates joint paths. Separate payoff classes represent the one-asset put, geometric basket put and max-call. The two regressors expose the same `fit` and `predict` interface, allowing `LSMPricer` to use one backward recursion and one forward evaluation for both methods. The neural model is an `nn.Module` with two `nn.Linear` layers and a leaky ReLU between them. PyTorch stores the linear weights transposed relative to the row-vector convention in equation (21); its forward operation implements the same affine map.

Polynomial fitting never stores the full design matrix. If Φ_k and y_k contain the rows of a block of paths, the normal equations are accumulated as

$$\Phi^\top \Phi = \sum_k \Phi_k^\top \Phi_k, \quad \Phi^\top y = \sum_k \Phi_k^\top y_k,$$

so memory depends on q and on the block size, not on the number of paths. At $d = 50$ and degree two, Φ would occupy about 1.1 GB for 10^5 rows in double precision, whereas $\Phi^\top \Phi$ occupies 14 MB. The columns are rescaled so that $\Phi^\top \Phi$ has a unit diagonal, and the system is solved through the Cholesky factorization $\Phi^\top \Phi = LL^\top$. Forming $\Phi^\top \Phi$ squares the condition number, $\kappa_2(\Phi^\top \Phi) = \kappa_2(\Phi)^2$, so rounding errors can be amplified; a QR factorization of Φ avoids this, at a higher cost. The factorization fails if the matrix is not numerically positive definite, and the condition number of the rescaled matrix is recorded at each date. Rather than relying on a general threshold, a check compares this solution with a direct least-squares solve based on a singular value decomposition of Φ itself, on the bases with the largest condition numbers observed in the study: the one-asset quartic, the five-asset bases of degrees four to six and the

fifty-asset quadratic. The check uses in-the-money states at the first and last regression dates, with 10^4 paths, the smallest training budget, where conditioning is worst, and compares both the predictions and the resulting exercise decisions. The predictions agree to within 3×10^{-10} , and no exercise decision changes.

Predictions are evaluated in blocks, and neural training processes mini-batches. Paths use float32 storage; polynomial solves, discounted cashflows and Monte Carlo sums use float64. Results are saved after every repetition, so that an interrupted study resumes and tables and figures are rebuilt without recomputation.

The notebook contains executable checks with distinct purposes:

- the eight-path example of Longstaff–Schwartz, using its published paths as reference data, checks the exercise recursion;
- recovery of a known polynomial, fitted over several blocks, checks the feature matrix, the solver and the monetary scaling, while network shapes and parameter counts check its interface;
- forward application on training paths is compared with the backward value, and each retained cashflow with the payoff at its stored exercise index;
- simulated log-return moments are inspected, and a European CRR put is compared with the Black–Scholes formula;
- on in-the-money states of the most ill-conditioned bases, at the first and last regression dates, the blockwise Cholesky solution is compared with a direct least-squares solve.

5.2 Contracts and reference values

The contracts are those of Lapeyre–Lelong [2, Tables 1, 3 and 9–11]: a one-dimensional put, a geometric basket put with a scalar benchmark, and dividend-paying max-calls. Parameters are shared across assets within each case, with equicorrelation ρ .

Contract	d	S_0^j	K	r	δ	σ	ρ	(T, N)
Put	1	100	110	.10	0	.25	0	(1, 10)
Geometric put	10	100	100	.05	0	.20	.20	(1, 10)
Max-call	5	100	100	.05	.10	.20	0	(3, 9)
Max-call	50	100	100	.05	.10	.20	0	(3, 9)

The polynomial candidates have degrees two to four for the put, two and three for the geometric basket, two to six for the five-asset max-call (six being the highest degree reported by Lapeyre–Lelong for this contract), and one and two at fifty dimensions, where the cubic basis would require 23426 coefficients.

For the put, Lapeyre–Lelong use the benchmark 11.987, a numerical value reported as accurate to the digits shown. A Cox–Ross–Rubinstein (CRR) tree is also computed: it refines the transition mesh but allows exercise only on the Bermudan dates, so refining it approximates the same contract. Values with 500, 1000 and 2000 substeps per exercise interval show the effect of refinement.

For the geometric basket $G_t = (\prod_{j=1}^d S_t^j)^{1/d}$, averaging log-prices gives a scalar geometric Brownian motion with

$$\sigma_G^2 = \frac{1}{d^2} \sigma^\top \Gamma \sigma, \quad \delta_G = \frac{1}{d} \sum_j \delta_j + \frac{1}{2d} \sum_j \sigma_j^2 - \frac{1}{2} \sigma_G^2.$$

Thus $dG_t = (r - \delta_G)G_t dt + \sigma_G G_t dB_t$. Under these constant coefficients, the conditional law of future basket increments depends only on the present basket, so observing the full asset vector adds no exercise information. The reduction is exact; the same CRR tree then provides a numerical reference, while the regressors still receive the ten asset prices.

No scalar reduction exists for the max-call. Lapeyre–Lelong quote the 95% intervals [26.14, 26.17] for five assets and [69.56, 69.95] for fifty assets, obtained by Becker et al.; they are not recomputed here. The prices tabulated by Lapeyre–Lelong are estimated on the training paths, which can be optimistic; they are points of comparison, not estimates directly comparable with independent test values.

5.3 Independent comparisons and uncertainty

Each repetition uses independent training, validation and test samples, shared between regression candidates: $M_{\text{train}} = 10^5$, $M_{\text{valid}} = 10^5$ and $M_{\text{test}} = 10^6$ paths, the test paths being generated and evaluated in blocks of 10^5 . There are $R = 5$ repetitions.

Only training paths determine the fitted regressors and the time-zero decision. At each date, validation paths provide cashflow targets under the already learned future policy, permitting a held-out mean squared error. These targets are independent of fitting that policy, but their variability includes irreducible cashflow noise. Different policies may also produce different targets; their MSEs are diagnostics rather than a direct ranking of policy values.

Within each family, the candidate with the highest validation policy value is selected before any test path is simulated. The comparison of interest is between the selected polynomial and the selected network; test values are also reported for every candidate. For one frozen policy, an approximate conditional 95% Monte Carlo interval is

$$\hat{V} \pm 1.96 \frac{s_H}{\sqrt{M_{\text{test}}}}, \quad s_H^2 = \frac{1}{M_{\text{test}} - 1} \sum_m (H^{(m)} - \hat{V})^2.$$

For a comparison on common test paths, form $D^{(m)} = H_{\text{neural}}^{(m)} - H_{\text{polynomial}}^{(m)}$ and use $s_D / \sqrt{M_{\text{test}}}$ for the standard error of their mean difference. Ignoring this pairing would discard the covariance between the two payoffs.

These intervals treat the fitted policies as fixed. Retraining on new paths changes the policies, and hence their values. If Δ_r denotes the mean paired difference in repetition r , the comparison across repetitions is summarized by

$$\bar{\Delta} \pm t_{R-1, 0.975} \frac{s_{\Delta}}{\sqrt{R}}, \quad s_{\Delta}^2 = \frac{1}{R-1} \sum_{r=1}^R (\Delta_r - \bar{\Delta})^2.$$

This interval already contains the test noise of each Δ_r , so the within-repetition standard errors are reported separately rather than added to it. With five repetitions it is approximate. No interval here bounds the shortfall from V_0 .

Fitting times include the validation diagnostics but not the simulation of the training and validation paths. The evaluation time, recorded separately and not reported here, includes the simulation of the test blocks. These are indicative measurements on one machine, not benchmarks.

6 Numerical results

6.1 Checks and reference values

All checks of Section 5.1 pass. The eight-path example returns 0.114434, the published value 0.1144, with the published exercise dates. The European CRR price differs from the Black–Scholes formula by 8×10^{-4} with 200 substeps per interval, and the simulated log-return correlation is 0.396 for a target of 0.4. The reference trees are stable under refinement:

Substeps per exercise interval	500	1000	2000
Put	11.98798	11.98735	11.98748
Geometric put	2.92972	2.92969	2.92975

The put reference agrees with the value 11.987 used by Lapeyre–Lelong. Rerunning the five-asset case with additional candidates reproduced the values of the common candidates to the digits reported.

6.2 Policy values with 10^5 training paths

Figure 2 shows the independent test value of every candidate, and Table 1 compares the policies selected by validation in each family.

Contract	Reference	Polynomial	Network	Network – polynomial
Put	11.987	11.988	11.901	-0.088 ± 0.074
Geometric put	2.930	2.915	2.886	-0.030 ± 0.011
Max-call, $d = 5$	[26.14, 26.17]	26.032	25.891	-0.141 ± 0.072
Max-call, $d = 50$	[69.56, 69.95]	68.907	68.761	-0.145 ± 0.061

Table 1: Selected policies with $M_{\text{train}} = 10^5$: mean test values over five repetitions and paired differences with 95% Student half-widths. The mean within-repetition Monte Carlo standard errors of the differences are 0.004, 0.002, 0.008 and 0.008.

For the put, degrees three and four reach the reference within Monte Carlo error. The networks are lower and less stable across repetitions; the width-128 network varies widely, and validation retained the width-32 network in four repetitions out of five. For the geometric basket, the full-state quadratic polynomial lies 0.014 below the tree, and the cubic basis, with 286 coefficients, is already worse.

For the five-asset max-call, the test value increases with the degree up to four and then decreases,

$$25.920, \quad 25.983, \quad 26.042, \quad 26.024, \quad 25.965 \quad \text{for degrees } 2, \dots, 6,$$

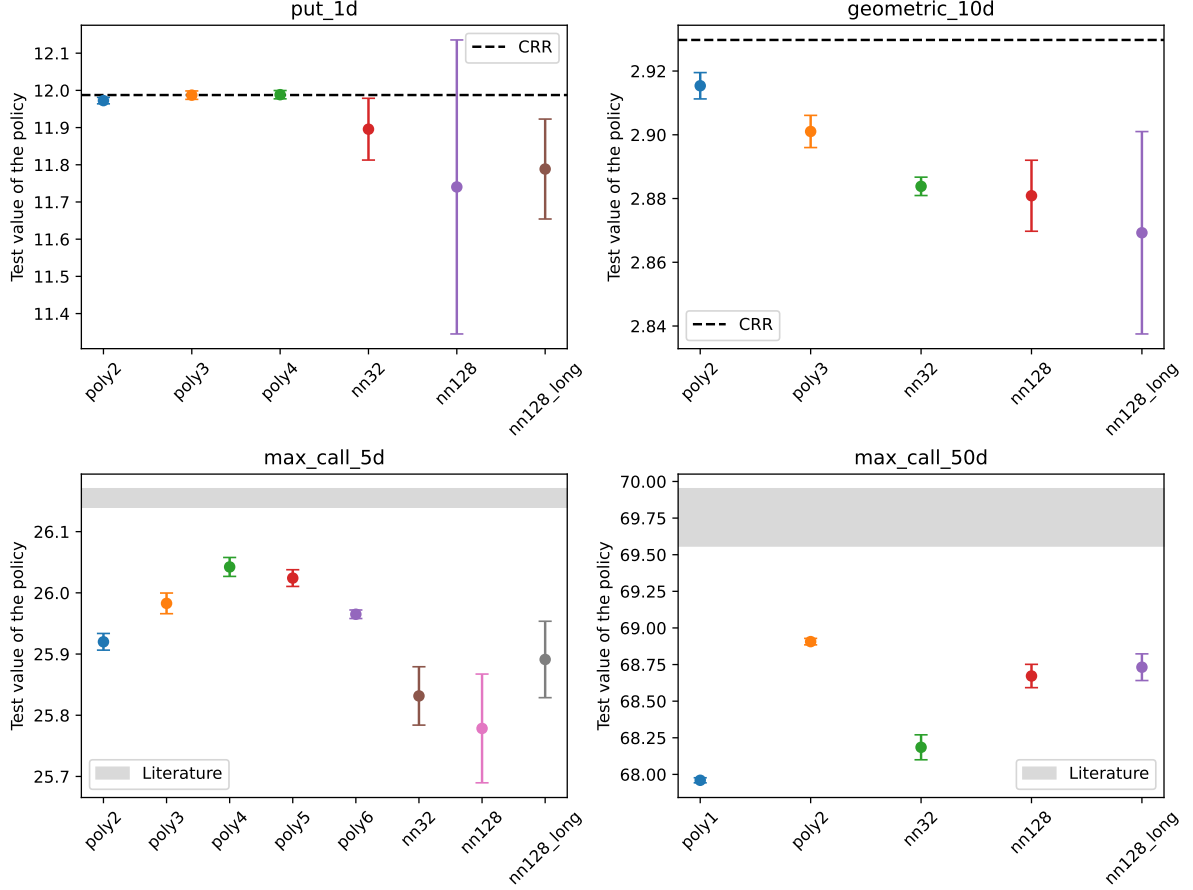


Figure 2: Test values of all candidates with $M_{\text{train}} = 10^5$: means over five repetitions and 95% Student half-widths. Dashed lines: CRR references; grey bands: literature intervals.

while the training estimates keep increasing: 26.19, 26.26 and 26.36 for degrees four to six. The best degree is therefore inside the candidate set. At fifty assets, the quadratic basis, the largest one available, remains ahead of every network.

In every contract the selected polynomial is ahead of the selected network, and the interval across repetitions excludes zero, narrowly for the put. For the max-calls, both families remain below the literature intervals, by about 0.1 at five assets and 0.65 at fifty. The expected values of these policies are lower bounds for V_0 , and the Monte Carlo means estimate them; without an upper bound, the remaining gap to V_0 cannot be split between the regressions and the reference.

6.3 Training estimates and training effort

Figure 3 shows the paired differences of each repetition and the mean gap between training estimates and test values of the selected policies. The gaps are larger in high dimension: 0.012 and 0.013 for the put, 0.025 and 0.027 for the geometric basket, 0.20 and 0.14 for the polynomial and the network at five assets, 0.31 and 0.48 at fifty. Their intervals across repetitions appear in Table 2; those of the put are compatible with zero. At fifty assets the ranking reverses between the two evaluations: the longer-trained network has a training estimate of 69.30 against 69.21 for the quadratic polynomial, but test values of 68.73 against 68.91. A comparison on training paths would have selected the wrong family. In the first repetition at fifty assets, the polynomial

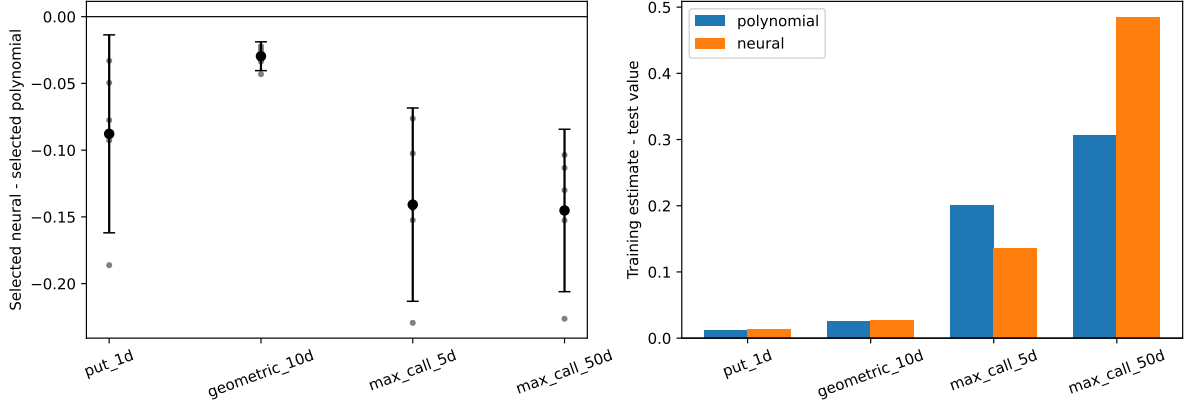


Figure 3: Left: selected network minus selected polynomial, per repetition (grey) and mean with 95% Student interval. Right: training estimate minus test value of the selected policies, $M_{\text{train}} = 10^5$.

also has the lower held-out regression error at every date. This is a descriptive observation, not evidence by itself: the two policies induce different targets, and a lower error may also reflect less variable cashflows. Only the test values decide.

The variant trained for five epochs after each warm start, with about three times as many updates, improves the five-asset max-call in all five repetitions, by 0.11 on average. At fifty assets the average change is +0.06 with 10^5 paths and +0.004 with 10^6 , but the differences take both signs across repetitions and reach 0.32 in absolute value with 10^6 paths. Additional epochs thus help in the five-asset case; elsewhere their effect is not resolved, and the network value varies substantially from one training to another. Larger samples also bring more updates per epoch, so the two effects cannot be separated here; the number of updates is only one aspect of training, together with the learning rate, the batch size and the initialization.

6.4 Effect of the training budget

The whole study was repeated with 10^4 training paths, and the fifty-asset case with 10^6 , keeping 10^5 validation and 10^6 test paths.

Contract	M_{train}	Polynomial	Network	Net. – poly.	Gap poly.	Gap net.
Put	10^4	11.985	11.904	-0.081 ± 0.033	-0.014 ± 0.076	-0.040 ± 0.087
	10^5	11.988	11.901	-0.088 ± 0.074	$+0.012 \pm 0.015$	$+0.013 \pm 0.018$
Geometric put	10^4	2.872	2.850	-0.022 ± 0.038	$+0.145 \pm 0.027$	$+0.044 \pm 0.039$
	10^5	2.915	2.886	-0.030 ± 0.011	$+0.025 \pm 0.014$	$+0.027 \pm 0.019$
Max-call, $d = 5$	10^4	25.887	25.858	-0.030 ± 0.049	$+0.125 \pm 0.356$	$+0.046 \pm 0.399$
	10^5	26.032	25.891	-0.141 ± 0.072	$+0.200 \pm 0.139$	$+0.136 \pm 0.112$
Max-call, $d = 50$	10^4	67.927	68.683	$+0.756 \pm 0.126$	$+0.205 \pm 0.151$	$+0.820 \pm 0.490$
	10^5	68.907	68.761	-0.145 ± 0.061	$+0.306 \pm 0.064$	$+0.484 \pm 0.165$
	10^6	68.993	68.823	-0.170 ± 0.159	$+0.018 \pm 0.018$	$+0.120 \pm 0.035$

Table 2: Selected policies for several training budgets: mean test values, paired differences with 95% Student half-widths, and mean gaps between training estimates and test values. All intervals are 95% Student half-widths across the five repetitions.

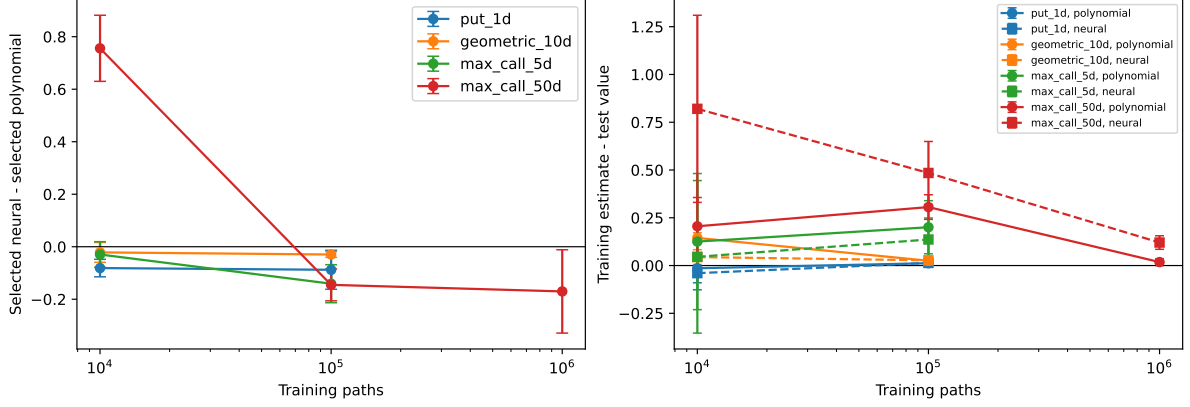


Figure 4: Effect of the number of training paths on the selected difference (left) and on the gap between training estimates and test values (right).

With 10^4 paths at fifty assets, the quadratic basis has about seven observations per coefficient. Its training estimate reaches 71.09, above the literature interval, while its test value is 67.73. Validation therefore retains the linear basis, and the network leads by 0.76. From 10^5 paths the quadratic basis can be estimated and the polynomial leads again; with 10^6 paths it gains 0.09, the network 0.06, and the difference remains negative, although its interval nearly reaches zero. The same effect appears at five assets with 10^4 paths: from degree two to six, the training estimates rise from 26.01 to 27.66 while the test values fall from 25.89 to 25.13, and validation retains degree two.

The gaps of Table 2 are themselves estimates, summarized like the other quantities by their dispersion across repetitions. The policy is fitted on the training paths, so a standard error computed on those same paths would not be justified. The intervals across repetitions avoid this; with five repetitions they remain approximate. The gaps of the put at both budgets, and of the five-asset max-call with 10^4 paths, are compatible with zero: these experiments do not exhibit a bias, which is not the same as showing that there is none. The gaps of the geometric basket and of the fifty-asset max-call are clearly positive.

Conditionally on the training data, the test average is unbiased for the value of the selected policy, so the expected gap equals the optimism of the training estimate; the suboptimality of the policy affects both evaluations and cancels. In fifty dimensions, the gap of the selected polynomial follows the ratio q/M between the number of coefficients and the number of paths: +0.018 at $q/M = 0.0013$, +0.205 at 0.0051 and +0.306 at 0.0133. This is an observation, not an isolated effect: the selected degree and the budget change together.

Computational cost. Median fitting times, including the validation diagnostics, illustrate the scaling. For the put, a polynomial takes 0.4 seconds and a network 10 to 35 seconds. At fifty assets with 10^5 paths, the quadratic polynomial takes 96 seconds and the networks 20 to 59 seconds; with 10^6 paths, 300 seconds against 56 to 155 seconds. The polynomial is much cheaper in low dimension, while its cost grows faster with the dimension. These are measurements on one laptop CPU, not benchmarks.

7 Conclusion

The Longstaff–Schwartz recursion was implemented with two interchangeable regressors and evaluated on independent paths. Its polynomial version reproduces the published eight-path example and the one-dimensional reference value. Because every learned policy has a value below V_0 , the two regressors were compared through their test values, with paired differences on common paths and five independent repetitions.

With 10^5 training paths and more, the policy selected among polynomial bases is ahead of the policy selected among one-hidden-layer networks in all four contracts, by 0.03 to 0.17, including at fifty assets where the quadratic basis is the largest feasible one. The ranking depends on the amount of data: with 10^4 paths at fifty assets, the quadratic basis cannot be estimated reliably and the network leads. Estimates computed on training paths are optimistic in high dimension and with little data, and can reverse the ranking of the two families; independent evaluation is therefore essential. Additional epochs after each warm start improved the five-asset network; at fifty assets their effect was not resolved.

These conclusions concern the tested recipes. Deeper networks, used in part by Lapeyre–Lelong, and other training schedules were not explored, and the network values vary markedly from one training to another. Five repetitions give approximate intervals, and the timings are indicative. For the max-calls, no upper bound was computed, so the gap to the literature intervals, about 0.1 at five assets and 0.65 at fifty, remains unexplained. Dual upper bounds and deeper or better trained networks are the natural extensions.

References

- [1] F. A. Longstaff and E. S. Schwartz (2001). Valuing American options by simulation: a simple least-squares approach. *The Review of Financial Studies*, 14(1), 113–147.
- [2] B. Lapeyre and J. Lelong (2020). Neural network regression for Bermudan option pricing. *arXiv:1907.06474v3*. <https://arxiv.org/abs/1907.06474v3>.
- [3] D. P. Kingma and J. Ba (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*. <https://arxiv.org/abs/1412.6980>.